PRODUCT REQUIREMENTS DOCUMENT

Machine Learning Framework for Microplastic Detection in Water

Development Blueprint for Antigravity + Google Colab

B.Tech Final Year Major Project — Phase 1

Software-only computer-vision system for microscopic water-sample image analysis.

Document	PRD / Development Specification
Primary ML environment	Google Colab
ML framework	Ultralytics YOLOv8
Dataset source	Roboflow-compatible annotated dataset
Backend	Python + FastAPI
Frontend	React + Vite
Primary output	Detection, classification, count, confidence, analysis, dashboard, report

Version: 1.0 | Prepared for implementation

1. Document Purpose

This PRD defines the complete functional, technical, data, machine-learning, backend, frontend, testing, and deployment requirements for developing the project. It is intended to be used as the master specification when building the system in Antigravity and training/testing the ML model in Google Colab.

2. Project Overview

The project proposes an AI-based framework that analyzes microscopic images of water samples to detect and classify visually annotated microplastic particles. Computer vision preprocessing is followed by YOLOv8 object detection. The system reports particle locations, classes, confidence scores, counts, and approximate dimensions when calibration is available. Results are presented through a web dashboard and can be exported as a PDF report.

3. Problem Statement

Traditional microplastic analysis can involve manual microscopic examination, which is time-consuming and labor-intensive. The proposed system aims to automate image-based detection and analysis to reduce manual effort and provide consistent digital results.

4. Objectives

• Develop an AI-based system for detecting microplastic particles from microscopic water-sample images.

• Automatically detect and classify particles using a trained YOLOv8 model.

• Support particle categories present in the selected annotated dataset.

• Estimate particle count and bounding-box dimensions.

• Provide confidence scores for individual detections.

• Provide a project-defined contamination assessment based on available image/sample information.

• Visualize results through an interactive dashboard.

• Generate an automated analysis report.

• Evaluate the trained detector using precision, recall, mAP50 and mAP50-95.

5. Scope

In Scope	Out of Scope / Future
Microscopic image upload	Direct chemical identification of polymer type
Image preprocessing	Satellite-image particle detection
YOLOv8 object detection	Real-time hardware microscope integration in Phase 1
Class prediction	Scientifically validated contamination thresholds without required sample metadata
Particle counting	Guaranteed physical size without microscope calibration
Confidence scores	Definitive pollution-source attribution
Dashboard and PDF report	Forecasting without sufficient historical/time-series data

6. Scientific and Usage Assumptions

• The model learns visual patterns from annotated training images; it does not chemically confirm that a detected particle is plastic.

• Where chemical confirmation is unavailable, UI/report wording should use 'detected' or 'suspected microplastic particles' rather than claiming definitive polymer identification.

• Physical dimensions in micrometers require microscope calibration such as a valid micrometers-per-pixel factor.

• If only image data are available, contamination assessment is an image-level/project-defined indicator, not a laboratory concentration measurement.

• The exact dataset classes must be read from data.yaml rather than hard-coded.

7. Target Users

• Environmental monitoring personnel

• Water-quality laboratory users

• Researchers and students

• Academic project evaluators

• Organizations performing image-based microplastic screening

8. High-Level System Architecture

Input image → OpenCV preprocessing → YOLOv8 inference → class + bounding box + confidence → particle analysis → contamination assessment → dashboard + PDF report

9. Technology Stack

Layer	Technology	Purpose
Notebook / ML	Google Colab	Training, evaluation and experimentation
Programming	Python	ML and backend implementation
Deep learning	Ultralytics YOLOv8	Object detection
Computer vision	OpenCV	Image processing
Numerical/data	NumPy, Pandas	Analysis and result tables
Dataset	Roboflow	Annotated image dataset and YOLO export
Backend	FastAPI	Inference API and report endpoints
Frontend	React + Vite	Interactive dashboard
Charts	Recharts	Particle statistics visualization
Report	Python PDF library	Automated analysis report

10. Repository Structure

```
microplastic-detection/
■■■ ml/
■ ■■■ notebooks/
■ ■ ■■■ microplastic_yolov8_training.ipynb
■ ■■■ scripts/
■ ■ ■■■ train.py
■ ■ ■■■ evaluate.py
■ ■ ■■■ predict.py
■ ■■■ data/
■ ■ ■■■ README.md
■ ■■■ models/
■ ■■■ README.md
■■■ backend/
■ ■■■ app/
■ ■ ■■■ main.py
■ ■ ■■■ config.py
■ ■ ■■■ models/schemas.py
■ ■ ■■■ services/
■ ■ ■ ■■■ detector.py
■ ■ ■ ■■■ image_processor.py
■ ■ ■ ■■■ analyzer.py
■ ■ ■ ■■■ report_generator.py
■ ■ ■■■ utils/file_utils.py
■ ■■■ uploads/
■ ■■■ results/
■ ■■■ reports/
■ ■■■ requirements.txt
■■■ frontend/
```

```
■ ■■■ src/
■ ■ ■■■ components/
■ ■ ■■■ pages/
■ ■ ■■■ services/
■ ■ ■■■ App.jsx
■ ■ ■■■ main.jsx
■ ■■■ package.json
■ ■■■ README.md
■■■ README.md
■■■ .gitignore
```

11. Dataset Requirements

Use a Roboflow dataset containing microscopic microplastic images with object-detection annotations. The dataset must provide YOLO-compatible labels or be exported to YOLO format. Before training, the notebook must inspect data.yaml and automatically obtain the class names.

Dataset attribute	Requirement
Image	Microscopic water-sample / microplastic image
Annotation	Bounding box for each annotated particle
Class label	Class assigned by dataset annotation
Split	Train / validation / test
Format	YOLO-compatible image and label structure
Metadata	Use when available; calibration/sample metadata are optional

12. Google Colab ML Workflow

• Install dependencies.

• Import libraries.

• Check whether GPU is available.

• Access/download the Roboflow dataset.

• Locate and read data.yaml.

• Inspect train, validation and test directories.

• Check annotations for missing or malformed files.

• Display sample images with bounding boxes.

• Calculate class distribution.

• Load a pretrained YOLOv8 model.

• Train on the selected dataset.

• Save best.pt and last.pt.

• Evaluate the model.

• Generate confusion matrix and precision-recall results.

• Upload a new microscopic image.

• Run inference.

• Create detection DataFrame.

• Calculate count, confidence and bounding-box dimensions.

• Perform project-defined contamination assessment.

• Save annotated output.

13. Training Requirements

Parameter	Requirement
Model	YOLOv8 pretrained object detector
Dataset config	Read from data.yaml
Class names	Read dynamically from data.yaml
Compute	Google Colab GPU when available
Image size	Configurable
Epochs	Configurable based on dataset and compute
Confidence threshold	Configurable, e.g. 0.25
Output	best.pt, last.pt and training results

14. Evaluation Requirements

• Precision

• Recall

• mAP@50

• mAP@50–95

• Confusion matrix

• Precision-recall curve

• Validation prediction images

• Inference time where measurable

All performance values must come from actual model execution. No fabricated accuracy, precision, recall or mAP values may appear in the final report.

15. Inference Requirements

The user uploads a JPG, JPEG or PNG microscopic image. The system validates the file, runs YOLOv8 inference, draws bounding boxes, and returns the class, confidence and location of each detection.

16. Particle Analysis

Output	Definition
Total particles	Number of model detections above the selected threshold
Class count	Number of detections for each class
Confidence	Model confidence associated with each detection
Bounding-box width	Width in pixels unless calibration is available
Bounding-box height	Height in pixels unless calibration is available
Bounding-box area	Width × height in pixels²
Physical size	Only calculate if microscope calibration is supplied

17. Detection Data Schema

```
particle_id class_name confidence x1 y1 x2 y2 width_pixels height_pixels area_pixels
```

18. Contamination Assessment

The system may provide Low / Moderate / High as a configurable, project-defined image-level assessment based on detection counts. Thresholds must be stored in configuration and clearly described as demonstration thresholds unless scientifically validated. If sample volume, filtration area and other laboratory metadata are available, additional concentration calculations may be implemented.

19. Backend API

Endpoint	Method	Purpose
/health	GET	Verify backend availability
/model-info	GET	Return model and class information
/predict	POST	Upload image and run inference
/generate-report	POST	Generate analysis PDF

20. /predict Response

```
{ "total_particles": 0, "detections": [], "class_distribution": {}, "average_confidence": 0.0,
"contamination_level": "Low", "result_image_url": "" }
```

21. Frontend Requirements

• Project title and subtitle.

• Drag-and-drop/browse image uploader.

• Image validation and preview.

• Analyze button.

• Original and annotated image display.

• Summary cards: total particles, average confidence, contamination level and detected classes.

• Class-wise particle distribution chart.

• Detection table.

• Generate PDF Report button.

• Analysis history.

• Loading state while inference is running.

• Clear error state when backend is unavailable or inference fails.

22. Frontend Components

• ImageUploader

• ResultCard

• DetectionTable

• ParticleChart

• ResultImage

• ReportButton

• HistoryTable

23. Report Requirements

• Project title

• Analysis date/time

• Input image name

• Total detected particles

• Class-wise counts

• Average confidence

• Contamination assessment

• Annotated result image

• Detection table

• Pixel dimensions when calibration is unavailable

• Scientific disclaimer about image-based prediction and chemical confirmation

24. Functional Requirements

ID	Feature	Requirement
FR-01	Upload	System shall accept supported microscopic image formats.
FR-02	Validation	System shall reject unsupported or invalid files.
FR-03	Preprocessing	System shall prepare the input image for model inference.
FR-04	Detection	System shall run the trained YOLOv8 model.
FR-05	Classification	System shall return dataset-defined class labels.
FR-06	Confidence	System shall return confidence per detection.
FR-07	Counting	System shall count valid detections.
FR-08	Analysis	System shall calculate class distribution and bounding-box measurements.
FR-09	Visualization	System shall show annotated results and statistics.
FR-10	Report	System shall generate a PDF report.
FR-11	Health	Backend shall expose a health endpoint.
FR-12	History	Application shall retain analysis records for the configured session/storage design.

25. Non-Functional Requirements

• Usability: A student should be able to demonstrate the workflow without command-line interaction after startup.

• Reliability: Invalid uploads and missing model files should produce readable errors.

• Maintainability: ML, backend, analysis and frontend logic should remain modular.

• Reproducibility: Dataset path, model configuration and inference threshold should be configurable.

• Performance: Inference time should be measured rather than claimed.

• Security: API keys must not be committed to source control.

• Portability: Avoid machine-specific absolute paths.

• Transparency: Display actual model metrics and clearly distinguish predictions from validated measurements.

26. Error Handling

Condition	Expected behavior
Unsupported extension	Return validation error
Corrupt image	Return readable error
Model missing	Return server configuration error
No detections	Show zero-result state without crashing
Backend unavailable	Frontend shows connection error
Invalid API key	Dataset access error with setup guidance
Missing data.yaml	Stop training and explain dataset structure issue

27. Security Requirements

• Store Roboflow API keys in environment variables or private Colab configuration.

• Never place real API keys in GitHub.

• Validate uploaded files before processing.

• Restrict upload size according to the deployment environment.

• Avoid executing uploaded files as code.

• Do not expose internal filesystem paths in production API responses.

28. Testing Plan

Test	Expected result
Backend health	Returns healthy status
Valid image	Prediction result returned
Invalid file	Validation error
Missing model	Clear configuration error
No detection	Zero-result response
Multiple particles	All valid detections listed
Class distribution	Counts match detection table
Annotated image	Bounding boxes visible
PDF report	Report generated with actual results
Frontend connection	Dashboard receives API response
Large image	Handled without application crash
CORS	Frontend can call backend

29. Development Sequence

• Create repository structure.

• Prepare Roboflow dataset.

• Build Colab dataset inspection.

• Train YOLOv8.

• Evaluate model.

• Implement new-image inference.

• Implement particle analysis.

• Implement contamination assessment.

• Move inference into FastAPI.

• Build React dashboard.

• Connect React to FastAPI.

• Implement PDF reports.

• Implement history.

• Run full testing.

• Prepare documentation and demonstration.

30. Antigravity Development Prompt Strategy

Use Antigravity in stages rather than requesting the entire application in one uncontrolled generation. Each stage should be completed and tested before the next stage begins.

Prompt	Stage
Master prompt	Understand complete product requirements
Stage 1	Create architecture and folders
Stage 2A	Prepare Google Colab dataset workflow
Stage 2B	Train YOLOv8
Stage 3	Evaluate model
Stage 4	New-image inference
Stage 4B	Particle analysis
Stage 4C	Contamination assessment
Stage 5	FastAPI backend
Stage 6	React dashboard
Stage 7	Frontend/backend integration
Stage 8A	PDF reporting
Stage 9	Testing and debugging
Final audit	Check entire project

31. Suggested End-to-End Demonstration

• Start the FastAPI backend.

• Start the React frontend.

• Open the dashboard.

• Upload a microscopic water-sample image.

• Click Analyze.

• Show the annotated image.

• Explain each bounding box.

• Show class and confidence values.

• Show total particle count.

• Show class distribution chart.

• Explain size as pixels unless calibration is available.

• Show the project-defined contamination assessment.

• Generate the PDF report.

• Show model evaluation metrics from Google Colab.

32. Future Scope

• Forecasting future microplastic contamination using sufficiently large historical/time-series data.

• Pollution-source analysis as a research feature requiring validation.

• Real-time microscope/camera integration.

• Larger and more diverse datasets.

• Model comparison and optimization.

• Cloud/mobile deployment.

• Integration with laboratory metadata and calibrated microscopy.

• Chemical confirmation workflows such as spectroscopy where appropriate.

33. Limitations

• Detection quality depends on image quality and dataset quality.

• Small or overlapping particles may be difficult to detect.

• Visually similar non-plastic particles may produce false positives.

• Model performance depends on training data and annotation quality.

• Physical particle size cannot be reported reliably without calibration.

• Image-based detection does not by itself chemically confirm polymer composition.

• Project-defined contamination thresholds require scientific validation before real-world interpretation.

• Forecasting requires historical data and is not supported by a single image.

34. Acceptance Criteria

• A valid microscopic image can be uploaded.

• The trained YOLOv8 model loads successfully.

• Detections are returned with class, bounding box and confidence.

• Particle counts agree with the returned detection records.

• Class distribution is generated from actual detections.

• Annotated images are generated.

• Dashboard displays actual backend results.

• PDF report contains actual analysis results.

• Google Colab can reproduce training/evaluation with the configured dataset.

• No fabricated performance values appear anywhere in the project.

• The final application handles invalid input and missing-model conditions without crashing.

35. Required Final Deliverables

• Google Colab notebook (.ipynb)

• Trained best.pt model

• Dataset configuration/data.yaml reference

• ML training/evaluation scripts

• FastAPI backend

• React frontend

• requirements.txt

• package.json

• README.md

• Automated PDF report module

- Sample annotated detection output
- Model evaluation results
- Testing report

36. Final Implementation Checklist

- ■ Dataset selected and license recorded
- ■ data.yaml inspected
- ■ Classes verified
- ■ Train/validation/test splits verified
- ■ Sample annotations visualized
- ■ YOLOv8 trained
- ■ best.pt saved
- ■ Evaluation metrics generated
- ■ Inference tested on unseen image
- ■ Particle DataFrame generated
- ■ Count and class distribution verified
- ■ Calibration behavior verified
- ■ FastAPI API tested
- ■ React dashboard tested
- ■ PDF report tested
- ■ Error handling tested
- ■ No secrets committed
- ■ README completed
- ■ Final end-to-end demonstration completed

37. Academic Notes

This PRD is a development specification. Dataset-specific image counts, class names, and model-performance values must be populated from the exact dataset and actual training run used by the project. The system should not present generated or assumed metrics as experimental results.